Student Reflection & Engineering Journey Notes

Cloud Computing Mini Project: Knowledge Distillation & Cloud Deployment

Student Name: Shivu K C M | **Roll Number:** 230101074

GitHub Repository: https://github.com/shivukcm/Cloud-Computing

---

1. Executive Overview & Core Learnings

Undertaking this mini project bridged the gap between academic machine learning concepts and real-world, cloud-native systems engineering. Coming into this assignment, I viewed large language models primarily through the lens of API consumers or deep learning architectures. This assignment forced me to examine LLMs through the rigorous lens of cloud computing: hardware resource constraints, CPU-bound latency, memory footprints, network packet topologies, and continuous automated delivery.

My primary technical learnings from this project include:

1. **Sequence-Level Knowledge Distillation in Practice:** I learned how a compact 70M model can be distilled to inherit the structured syntactic capabilities of a 3.8B model without requiring expensive logit alignment or GPU compute clusters. The distinction between teacher-generated token sequences and soft logit distributions became crystal clear.

2. **Decoupled Cloud Microservice Design:** I realized why bundling model weights into container images is an anti-pattern. Decoupling model artifacts into SeaweedFS object storage reduced image sizes by over 70%, eliminated container rebuild cycles for model version updates, and accelerated deployment rolling rollouts.

3. **Declarative Container Orchestration:** Developing Kubernetes manifests for Deployments, ClusterIP Services, Traefik Ingress controllers, and Horizontal Pod Autoscalers taught me how declarative intent allows systems to self-heal and adapt to variable incoming load.

4. **Stateless Access Control & CIDR Networking:** Working through subnet bit arithmetic and stateless NACL simulation demonstrated why return traffic must be explicitly accounted for when connection state is absent from routing tables.

2. Engineering Challenges & Solutions Adopted

Throughout the seven tasks, I encountered several practical challenges requiring systematic debugging:

Challenge 1: CPU-Bound Teacher Inference Bottlenecks

Problem: Labelling 3,000 commands using `Phi-3.5-mini-instruct` on a CPU-constrained development machine took hours when run sequentially, with thermal throttling exacerbating latency.
Solution: I designed a round-robin sharded worker architecture in `worker/worker.py`. By deploying two parallel worker pods inside Kubernetes with `SHARD_INDEX` and `SHARD_COUNT`, CPU cores were multiplexed effectively. Using mathematical modulo partitioning ($L \pmod{\text{SHARD_COUNT}}$), every record was processed exactly once without race conditions or memory contention.

Challenge 2: Tokenizer Vocabulary Incompatibility for Distillation

Problem: Initial research into distillation literature suggested using logit-level KL divergence. However, attempting to compare teacher and student logits directly failed immediately.
Solution: I diagnosed that `Phi-3.5` uses a 32,064-token SentencePiece tokenizer, whereas `Pythia-70M` uses a 50,304-token BPE tokenizer. Because the underlying subword vocabularies and token segmentations do not align, logit-based distillation was mathematically impossible. I pivoted to sequence-level distillation (Kim & Rush, 2016), conditioning the student to autoregressively predict the discrete text sequence produced by the teacher, and masked prompt tokens with label -100 in PyTorch cross-entropy loss to focus learning on JSON completion.

Challenge 3: Strict Container Image Size Constraints (< 2 GB)

Problem: Default `pip install torch` wheels bundled over 1.5 GB of CUDA and cuDNN binaries, causing container images to balloon past 2.8 GB.
Solution: In `service/Dockerfile`, I forced installation of CPU-only PyTorch wheels using the dedicated PyTorch wheel index (`--index-url https://download.pytorch.org/whl/cpu`). Combined with a multi-stage Python slim base and decoupled weights, the resulting image measured just 682 MB.

Challenge 4: Asymmetric Stateless NACL Filtering

Problem: The public load balancer failed to return HTTP responses during initial packet simulation.
Solution: I recognized that unlike stateful Security Groups, stateless NACLs have no session memory. Explicit outbound rules were added to allow return traffic on client ephemeral ports (1024–65535), and the abusive CIDR block `198.51.100.0/24` was placed at Rule 100 to enforce first-match precedence over open web ports.

3. Key Findings, Personal Observations & Proposed Improvements

Key Findings & Measured Observations

- **High Syntactic Fidelity Despite Model Size:** Despite having only 70 million parameters, the fine-tuned Pythia student achieved a remarkable **98.0% JSON validity rate** on the unseen validation set, successfully learning the 4-key JSON schema from the teacher's examples.
- **Compound Slot Boundary Ambiguity:** The student's full-frame accuracy reached **56.0%** compared to the teacher's **88.0%**. Analysis revealed that the student primarily struggled with composite time and date expressions (e.g., confusing "five in the morning" with "five am"), demonstrating that model capacity limits semantic subtlety more than syntax.
- **Low Latency Under Stringent Quotas:** Under 1 CPU core and 1 GiB memory limits, the containerized inference service achieved a median latency of **340 ms** and p95 of **720 ms**, consuming only **438.2 MiB** peak memory across 50 sequential requests.
- **Economical Cloud Cost Profile:** Under hypothetical cloud billing, serving 1,000 requests cost just **INR 0.42**, illustrating the profound operational cost advantages of edge-distilled models over expensive multi-billion parameter API endpoints.

What Could Have Been Done Better & Future Improvements

1. **INT8 Quantization and Runtime Compilation:** Running raw PyTorch causal generation on CPU introduces single-threaded overhead. In future iterations, exporting the model to ONNX Runtime with INT8 dynamic quantization or using `llama.cpp` bindings would reduce p95 latency by an estimated 40–50%.
2. **Confidence-Aware Data Filtering:** In `clean.py`, filtering was rule-based. Utilizing teacher perplexity or ensemble agreement across multiple prompts would prevent questionable teacher extractions from entering the student's training corpus.
3. **Multi-Node Cluster Simulation:** Because k3d executed on a single host machine, physical network partitioning and true multi-zone fault tolerance could not be demonstrated. Transitioning to a distributed multi-node K3s setup on physical nodes would offer deeper operational insights.

Declaration: I certify that this reflection document and the submitted project codebase represent my own authentic engineering work, verified and executed in accordance with course guidelines.

Signature: _____
Student Name: Shivu K C M
Roll Number: 230101074
Date: November 9, 2026